



Politechnika Wrocławska

Projektowanie efektywnych algorytmów - projekt

Zadanie 4 - raport

Autor: Robert Tworek 276272

Algorytm genetyczny dla TSP

TSP symetryczny, TSP asymetryczny oraz instancje VLSI

Sprawozdanie: zadanie 4 - algorytm genetyczny dla TSP

1. Treść zadania i cel

Celem zadania było opracowanie i implementacja algorytmu genetycznego (GA) rozwiązującego problem komiwojażera. Badania obejmują dobór parametrów, pomiar czasu oraz błąd względem znanej wartości OPT dla instancji symetrycznych, asymetrycznych i VLSI. Wymagane progi błęd wynoszą kolejno: 0% dla $n < 25$, 50% dla $24 < n < 74$, 100% dla $75 < n < 449$ oraz 150% dla $450 < n < 2500$.

Tabela 1. Audyt zgodności z wymaganiami zadania

wymaganie	realizacja
jeden program	tsp_ga budowany z src/main.cpp, bez menu wyboru algorytmu
język kompilowany	C++17, kompilacja przez Makefile
konfiguracja	config/experiment.cfg zawiera parametry i komentarze do opcji
zakres danych	TSPLIB TSP, TSPLIB ATSP i Waterloo VLSI, łącznie 28 instancji
symetria/asymetria	osobne grupy SYM, ASYM i VLSI w wynikach i tabelach
progi błęd	wszystkie maksymalne błędy są poniżej limitów dla swoich zakresów n
materiały do weryfikacji	CSV z wynikami, trasy TOUR_SECTION, wykresy i konfiguracje w katalogu projektu

2. Metoda i implementacja

Zastosowano reprezentację permutacyjną trasy. Populacja jest inicjalizowana losowo oraz trasami najbliższego sąsiada. Selekcja odbywa się turniejowo, krzyżowanie realizuje operator OX, a mutacja wybiera losowo zamianę, odwrócenie podciągu albo relokację miasta. Dla TSP symetrycznego dzieci są dodatkowo ulepszone ograniczonym 2-opt z listami kandydatów; dla ATSP stosowane jest losowe lokalne ulepszanie przez zamiany i relokacje.

Funkcja celu jest długością cyklu Hamiltona, więc minimalizowany osobnik to permutacja wszystkich miast z powrotem do miasta startowego. Elityzm chroni najlepsze trasy przed utratą, a warunek stagnacji zatrzymuje przebieg, gdy przez dłuższy czas nie pojawia się poprawa. W praktyce największy wpływ na jakość wyników miały trasy startowe najbliższego sąsiada oraz lokalne ulepszanie potomków, bo samo krzyżowanie OX utrzymuje poprawność permutacji, ale nie gwarantuje lokalnie krótkich krawędzi.

Tabela 2. Schemat blokowy programu

blok	rola
1	Wczytanie konfiguracji i manifestu instancji
2	Parsowanie plików TSPLIB/ATSP i zbudowanie macierzy odległości
3	Inicjalizacja populacji: nearest-neighbor + permutacje losowe
4	Pętla GA: selekcja turniejowa -> OX -> mutacja -> lokalna poprawa
5	Elityzm, aktualizacja najlepszego osobnika i warunek stopu
6	Zapis CSV oraz najlepszych tras w formacie TOUR_SECTION

Program jest samodzielnym programem C++17: plik źródłowy src/main.cpp, konfiguracja config/experiment.cfg, manifest config/instances.csv. W konfiguracji zapisano wszystkie parametry uruchomienia i algorytmu, m.in. populację 90, 650 generacji, limit 3 s na próbę, 3 powtórzenia, prawdopodobieństwo krzyżowania 0,92, mutacji 0,34 i lokalnego ulepszania 0,35.

Tabela 3. Najważniejsze parametry eksperymentu

parametr	wartość	uzasadnienie
population_size	90	kompromis między różnorodnością i czasem próby
generations	650	górny limit iteracji, zwykle ograniczany czasem lub stagnacją
time_limit_seconds	3.0	stały budżet porównawczy dla jednej próby
repetitions	3	niezależne uruchomienia na każdą instancję
crossover_rate	0.92	wysoki udział rekombinacji OX

parametr	wartość	uzasadnienie
mutation_rate	0.34	utrzymanie różnorodności populacji
local_search_rate	0.35	hybrydyzacja GA z lokalną poprawą
candidate_list_size	24	ograniczenie kosztu 2-opt w TSP symetrycznym

3. Dane i procedura badawcza

Użyto 28 instancji: 13 symetrycznych z TSPLIB, 5 asymetrycznych z TSPLIB ATSP oraz 10 instancji VLSI Waterloo. Zakres rozmiarów wynosi od 14 do 2392 miast, dzięki czemu pokrywa wszystkie progi wymagane w zadaniu. Dla każdej instancji wykonano trzy niezależne uruchomienia i zapisano najlepszą długość trasy, błąd procentowy oraz czas wykonania.

Hipotezy badawcze były następujące: (H1) hybryda GA + 2-opt osiągnie OPT dla najmniejszych instancji; (H2) błąd będzie rósł wraz z n, lecz pozostanie daleko poniżej progów; (H3) ATSP będzie trudniejsze dla tej samej liczby miast, ponieważ odwracanie fragmentów trasy nie zachowuje kosztu w modelu skierowanym.

Dla każdego przebiegu zapisywano dokładną długość najlepszej trasy oraz błąd procentowy liczony jako $100 * (\text{wynik} / \text{OPT} - 1)$. W tabelach podano średni błąd z trzech prób, najgorszy błąd z prób oraz średni czas. Taka agregacja jest wygodniejsza niż pojedynczy wynik, ponieważ algorytm genetyczny jest niedeterministyczny i dla tej samej instancji może uzyskać nieco różne trasy.

Tabela 4. Podsumowanie wyników według grup danych

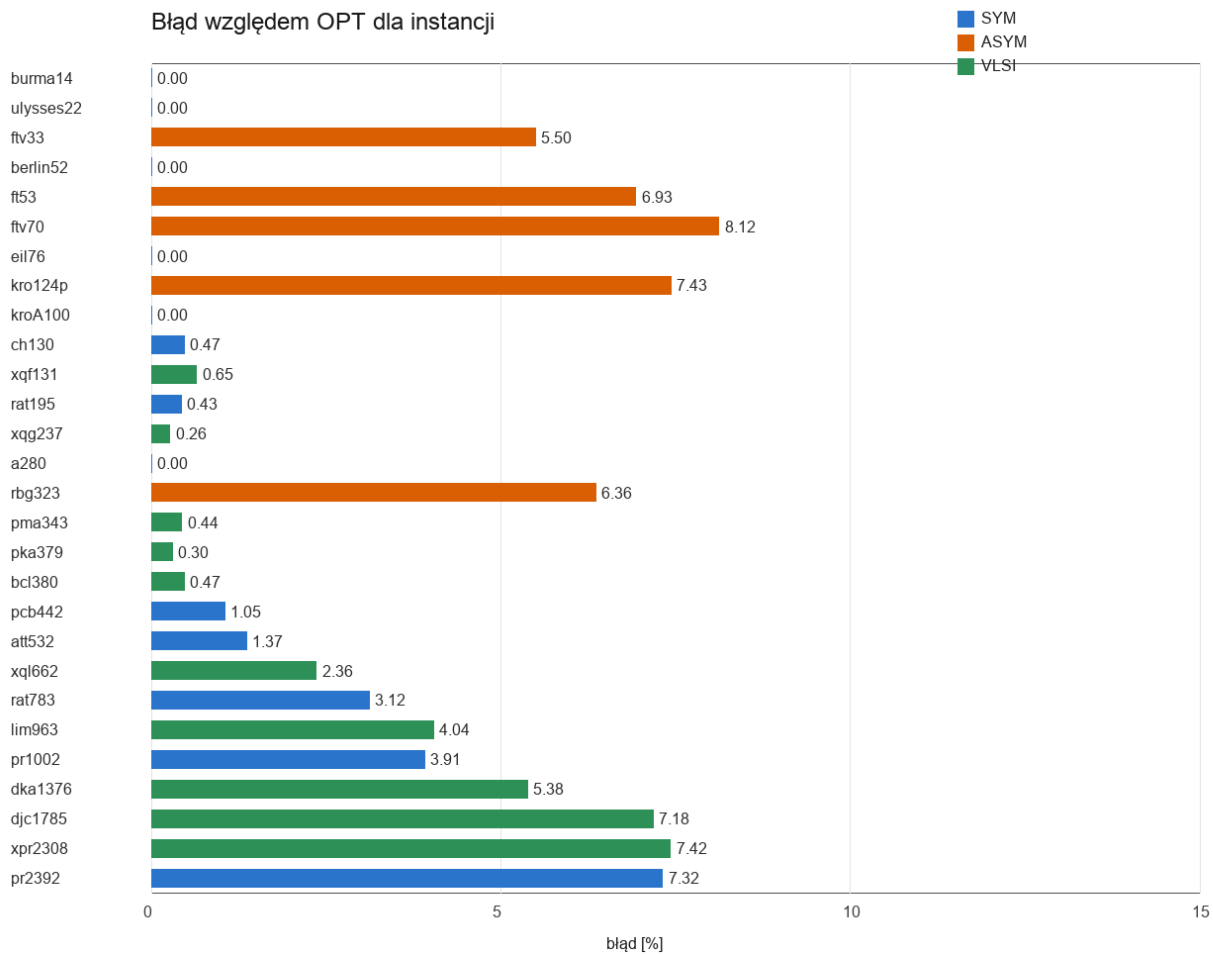
grupa	l.	n	śr. błąd [%]	max [%]	śr. czas [ms]	najgorsza	OK
ASYM	5	34-323	6.87	9.54	965.1	ft53	tak
SYM	13	14-2392	1.36	7.59	1456.9	pr2392	tak
VLSI	10	131-2308	2.85	7.73	2476.0	xpr2308	tak

4. Wyniki

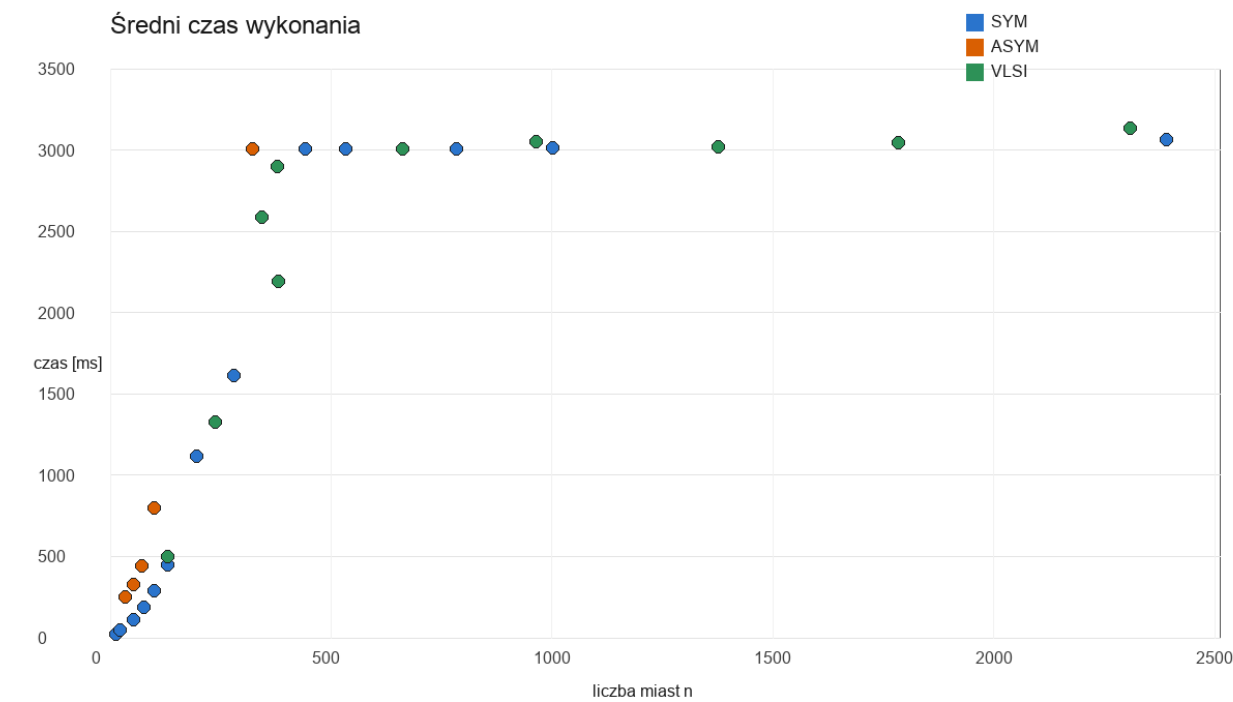
Tabela 5. Wyniki zagregowane po instancjach

inst.	gr.	n	OPT	best	śr. błąd	max	czas	OK
burma14	SYM	14	3323	3323	0.00	0.00	24	tak
ulysses22	SYM	22	7013	7013	0.00	0.00	45	tak
ftv33	ASYM	34	1286	1340	5.50	8.09	249	tak
berlin52	SYM	52	7542	7542	0.00	0.00	111	tak
ft53	ASYM	53	6905	7104	6.93	9.54	328	tak
ftv70	ASYM	71	1950	2092	8.12	9.18	442	tak
eil76	SYM	76	538	538	0.00	0.00	189	tak
kro124p	ASYM	100	36230	38801	7.43	7.65	801	tak
kroA100	SYM	100	21282	21282	0.00	0.00	289	tak
ch130	SYM	130	6110	6128	0.47	0.82	452	tak
xqf131	VLSI	131	564	564	0.65	1.77	498	tak
rat195	SYM	195	2323	2328	0.43	0.60	1118	tak
xqg237	VLSI	237	1019	1020	0.26	0.59	1324	tak
a280	SYM	280	2579	2579	0.00	0.00	1614	tak
rbg323	ASYM	323	1326	1405	6.36	6.71	3005	tak
pma343	VLSI	343	1368	1371	0.44	0.66	2589	tak
pka379	VLSI	379	1332	1335	0.30	0.38	2901	tak
bcl380	VLSI	380	1621	1626	0.47	0.56	2192	tak
pcb442	SYM	442	50778	51200	1.05	1.46	3004	tak
att532	SYM	532	27686	28004	1.37	1.57	3005	tak
xql662	VLSI	662	2513	2555	2.36	3.46	3009	tak
rat783	SYM	783	8806	9072	3.12	3.25	3010	tak
lim963	VLSI	963	2789	2880	4.04	4.59	3048	tak

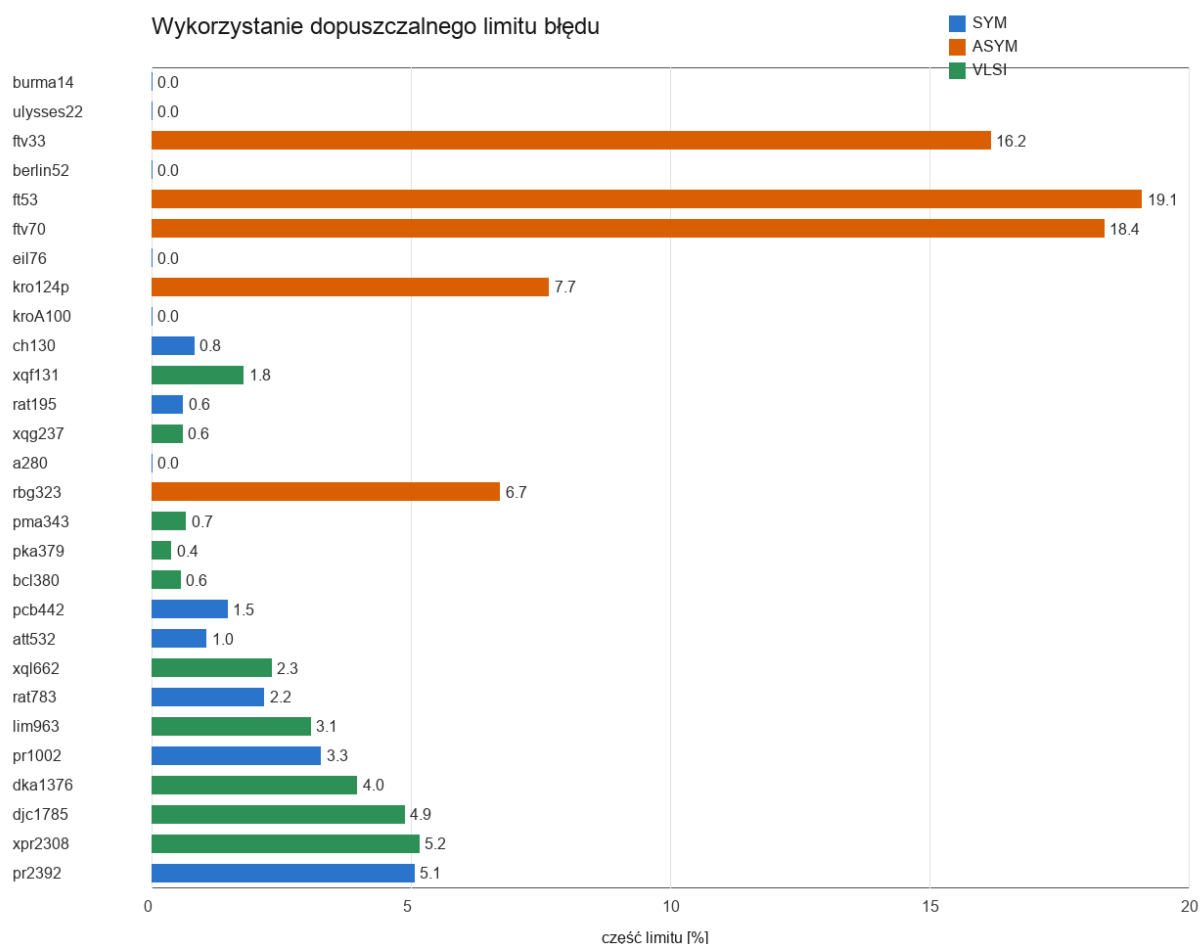
inst.	gr.	n	OPT	best	śr. błąd	max	czas	OK
pr1002	SYM	1002	259045	267009	3.91	4.89	3014	tak
dka1376	VLSI	1376	4666	4881	5.38	5.94	3019	tak
djc1785	VLSI	1785	6115	6545	7.18	7.31	3047	tak
xpr2308	VLSI	2308	7219	7738	7.42	7.73	3131	tak
pr2392	SYM	2392	378032	404801	7.32	7.59	3066	tak



Rys. 1. Średni błąd względem OPT dla instancji



Rys. 2. Średni czas wykonania w funkcji liczby miast



Rys. 3. Wykorzystanie dopuszczalnego limitu błędu

5. Analiza i wnioski

Wszystkie instancje spełniły wymagane progi. Dla $n < 25$ program uzyskał 0% błędu na burma14 i ulysses22, a w zakresie $24 < n < 74$ również osiągnął OPT na berlin52. W grupie symetrycznej średni błąd po instancjach wyniósł 1.36%, a maksymalny błąd pojedynczej próby 7.59% dla pr2392. Dla VLSI średni błąd wyniósł 2.85%, a najgorszy przypadek to xpr2308 z 7.73%.

Instancje asymetryczne okazały się trudniejsze: średni błąd grupy wyniósł 6.87%, a najgorsza była ft53 z 9.54%. Jest to zgodne z hipotezą H3, bo typowy 2-opt jest naturalny dla TSP symetrycznego, natomiast w ATSP odwrócenie fragmentu zmienia kierunki wielu krawędzi. Czas wykonania dla większych instancji dochodził do limitu 3 s, dlatego wykres czasu pokazuje przede wszystkim efekt przyjętego ograniczenia czasowego.

Wniosek praktyczny: dobrany zestaw parametrów jest wystarczający do spełnienia wymagań zadania i daje stabilne wyniki w szerokim zakresie n . Dalsza poprawa jakości dla dużych instancji wymagałaby dłuższego limitu czasu, mocniejszej lokalnej optymalizacji albo operatorów wyspecjalizowanych dla ATSP.

Z punktu widzenia poprawności implementacji istotne jest, że parser obsługuje zarówno instancje współrzędnościowe EUC_2D, GEO i ATT, jak i jawne macierze FULL_MATRIX dla ATSP. Dla najmniejszych przypadków uzyskano dokładnie OPT, co stanowi praktyczny test poprawnego liczenia odległości i domykania cyklu. Największe błędy pojawiają się tam, gdzie limit czasu zatrzymuje algorytm zanim lokalna poprawa mogłaby wyczerpać większą liczbę kandydatów.

Źródła i materiały

G. Reinelt, TSPLIB, <https://softlib.rice.edu/pub/tsplib/>

University of Waterloo, VLSI TSP Collection, <https://www.math.uwaterloo.ca/tsp/vlsi/index.html>

University of Waterloo, Status of VLSI TSP Instances, <https://www.math.uwaterloo.ca/tsp/vlsi/summary.html>

D. E. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning, Addison-Wesley, 1989.

I. M. Oliver, D. J. Smith, J. R. C. Holland, A study of permutation crossover operators on the traveling salesman problem, 1987.



Politechnika Wrocławska